

CASHALL PARTNER APP — COMPLETE API & BACKEND HANDOFF

Comprehensive Codebase Reverse-Engineering, API Inventory, Critical Security Audit & Implementation Specification

Repository:	https://github.com/cashall7003216788-jpg/CashALL	Date:	August 2026 (Live Audit)
Live Website:	https://www.cashall.in/	Target Audience:	Authorized Partner App Mobile Engineering Team
Database Engine:	Supabase PostgreSQL 15 (PgBouncer Port 6543)	Audit Scope:	All 65 API routes, 41 DB models, Auth & Security

Document Structure & Table of Contents

1. System Architecture	9. Supabase Database & RLS	17. Notification Workflow
2. Complete API Inventory (65 Routes)	10. Order State Machine	18. API Security Checklist
3. API Request / Response Reference	11. Partner App Workflow	19. Idempotency Implementation
4. Customer & Admin Authentication	12. Payment Architecture (UPI Manual)	20. Environment Variables Audit
5. Critical Security Audit (Agent Auth)	13. Upload & Storage Workflow	21. Credential Requirements Checklist
6. Production Environment Specification	14. Pickup Failed Workflow (New)	22. Existing vs Missing APIs
7. Staging Environment Specification	15. Order Received / Complete API	23. Required Backend Fixes
8. Database Models Full Audit	16. Bill / Invoice Generation	24. Partner App Dev Checklist

1. System Architecture Overview

The CashALL platform operates on a centralized backend architecture connecting the consumer website, admin operations, and the future Partner Mobile App:

- **Frontend Web:** Next.js 14 App Router (React 18, Tailwind CSS, Framer Motion) hosted serverless on Vercel.
- **Backend Framework:** Next.js API Routes (Node.js runtime) running serverless on Vercel.
- **Primary Database:** PostgreSQL 15 hosted on Supabase (Region: AWS Mumbai aws-0-ap-south-1, Project: jqysknhobtpcbyylnfc). Connected via Prisma ORM 5.21 through an IPv4 transaction pooler (PgBouncer) on port 6543.
- **Storage System:** Supabase Storage with 4 buckets: customer-devices, qc-reports, payment-screenshots, and payment-proofs.
- **Authentication:** Firebase Phone OTP Auth on frontend; verified server-side with firebase-admin; mapped to User table in PostgreSQL.
- **Notification Services:** Automated admin alerts via WhatsApp (CallMeBot API); transactional customer emails via Nodemailer (Gmail SMTP) & Resend; SMS handled directly by Firebase Phone Auth.
- **OCR Scanner:** Client-side Tesseract.js in agent/admin portals to parse 12-digit UTR from payment receipts.
- **Payment Gateway:** **Actual payment gateway integration not found.** RazorpayX and Cashfree are not integrated; payouts are conducted manually via corporate UPI handles (GPay/Paytm).

2. Complete API Inventory (Categorized Audit of All Implemented Endpoints)

Method	Endpoint Path	Purpose	Current Auth	Models Accessed	Implementation Status	App Safe?
POST	/api/v1/agent/login	Agent login (name/phone + pass)	Public	User	EXISTING — NEEDS FIX	YES
GET	/api/v1/agent/orders	Fetch assigned orders for agent	Public (Query Param)	Order, Pickup, Quote	CRITICAL SECURITY ISSUE	YES (After fix)
POST	/api/v1/orders/[id]/start-inspection	Transition to INSPECTION_STARTED	Bearer Token	Order, AuditLog	EXISTING — NEEDS FIX	YES
POST	/api/v1/agent/orders/[id]/inspection	Submit physical QC & IMEI	None (No Auth)	QcReport, Imei, Order	EXISTING — NEEDS FIX	YES
POST	/api/v1/agent/upload-payment	Upload payment screenshot & UTR	None (No Auth)	Payment, Order	EXISTING — NEEDS FIX	YES
POST	/api/v1/agent/orders/[id]/complete	Complete handover & trigger bill	None (No Auth)	Payment, Order	EXISTING — NEEDS FIX	YES
GET	/api/v1/orders/[id]/generate-bill	Generate structured bill JSON	Public / Optional Auth	Invoice, Order, Quote	EXISTING — WORKING	YES
POST	/api/v1/orders/[id]/device-received	Confirm device received	Bearer Token	Order, AuditLog	EXISTING — NEEDS FIX	YES
POST	/api/v1/orders/create	Customer create order & pickup	Public / Token	Order, Pickup, User	EXISTING — WORKING	INTERNAL / WEB
GET	/api/v1/orders/[id]	Track order details & timeline	Public / Token	Order, Quote, Address	EXISTING — WORKING	YES
GET	/api/v1/orders/user	List orders for logged-in user	Bearer Token	Order, Quote	EXISTING — WORKING	CUSTOMER ONLY
POST	/api/v1/orders/[id]/accept-offer	Customer accept revised offer	Bearer Token	Order, Offer	EXISTING — WORKING	CUSTOMER ONLY
POST	/api/v1/orders/[id]/decline-offer	Customer decline revised offer	Bearer Token	Order, Offer	EXISTING — WORKING	CUSTOMER ONLY
POST	/api/v1/orders/[id]/cancel	Customer/Admin cancel order	Bearer Token	Order, AuditLog	EXISTING — WORKING	CUSTOMER / ADMIN
POST	/api/v1/orders/[id]/final-offer	Issue revised price offer	Bearer Token	Offer, Order	EXISTING — WORKING	ADMIN / AGENT
POST	/api/v1/orders/[id]/verify-identity	Customer ID verification	Bearer Token	Verification, Order	PARTIALLY IMPLEMENTED	OPTIONAL
POST	/api/v1/orders/[id]/esign	Customer digital sale agreement	Bearer Token	Document, Order	PARTIALLY IMPLEMENTED	OPTIONAL
POST	/api/v1/quotes/calculate	Calculate buyback quote	Public	Quote, PricingRule	EXISTING — WORKING	YES
POST	/api/v1/quotes/save	Save quote to database	Public / Token	Quote	EXISTING — WORKING	YES
GET	/api/v1/catalog	Fetch brands, models, variants	Public Anon	Brand, DeviceModel	EXISTING — WORKING	YES
GET	/api/v1/catalog/questions	Fetch condition diagnostic questions	Public Anon	ConditionQuestion	EXISTING — WORKING	YES
GET	/api/v1/serviceability	Check PIN code coverage	Public Anon	ServiceArea	EXISTING — WORKING	YES
POST	/api/v1/storage/upload	Upload file to Supabase Storage	Public / Token	Supabase Storage	EXISTING — WORKING	YES
POST	/api/v1/notifications/register	Register FCM push token	Bearer Token	NotificationToken	EXISTING — WORKING	YES
POST	/api/v1/agent/orders/[id]/pickup-failed	Report failed doorstep pickup	N/A	N/A	NOT CURRENTLY IMPLEMENTED	NEW API REQUIRED

POST	/api/v1/admin/orders/[id]/assign-pickup	Admin assign agent to pickup	Admin Bearer	Order, Pickup	EXISTING — WORKING	ADMIN ONLY
-------------	---	------------------------------	--------------	---------------	---------------------------	------------

3. API Request / Response Reference (Actual Contracts)

Below are the exact HTTP contracts based on current production code for endpoints relevant to the Partner App:

Endpoint: POST /api/v1/agent/login	Purpose: Authenticates field logistics agent
Headers: Content-Type: application/json	Request Body / Parameters:
{ "inputName": "Wasim Shirazi", "cleanPhone": "8336856710", "inputPassword": "8336856710" }	Success Response (200): HTTP 200 OK { "success": true, "token": "tok_agent_7ab7cdc4-b48b-420f-8ac0-e0576ef973aa-1724982120000", "agent": { "id": "7ab7cdc4-b48b-420f-8ac0-e0576ef973aa", "name": "Wasim Shirazi", "email": "wasim.shirazi@cashall.in", "phone": "8336856710", "role": "AGENT" } } Error Response: HTTP 401 Unauthorized { "success": false, "error": "Invalid agent credentials. No AGENT account found matching these details." }
Endpoint: GET /api/v1/agent/orders?phone=8336856710	Purpose: Fetches assigned pickup tasks for agent (CURRENT: UNPROTECTED)
Headers: None currently checked (Accepts query param)	Request Body / Parameters:
No request body (GET)	Success Response (200): HTTP 200 OK { "success": true, "orders": [{ "id": "23a1ef90-50d4-4a55-8772-2bf2eb2659d4", "orderNumber": "#CA61962", "status": "PARTNER_ASSIGNED", "pickupDate": "2026-08-29", "pickupTimeSlot": "Today (1 PM - 4 PM)", "customerName": "Rounak Gupta", "customerPhone": "7003216788", "address": "123, 1, Gopal Lal Tagore Rd, Bonhooghly, Baranagar - 700036", "deviceName": "Samsung Galaxy S23 Ultra 5G (256 GB)", "estimatedPrice": 34310, "finalPrice": 34310, "urn": null }] } Error Response: HTTP 200 OK (With 0 orders) { "success": true, "orders": [] }
Endpoint: POST /api/v1/agent/orders/[id]/inspection	Purpose: Submits doorstep hardware inspection findings & 15-digit IMEI
Headers: Content-Type: application/json	Request Body / Parameters:
{ "imei": "864291048291048", "screenFinding": "Flawless screen, no burn-in", "bodyFinding": "Minor scratches on outer bezel", "revisedPrice": 32500, "reason": "Bezel scratch deduction", "customerEmail": "customer@gmail.com", "agentName": "Wasim Shirazi" }	Success Response (200): HTTP 200 OK { "success": true, "message": "Inspection & IMEI recorded successfully.", "order": { "id": "...", "status": "ACCEPTED", "finalPrice": 32500 }, "qcReport": { "id": "...", "imeiNumber": "864291048291048", "status": "APPROVED" } } Error Response: HTTP 400 Bad Request { "success": false, "error": "Valid IMEI number is required (min 5 digits)" }
Endpoint: POST /api/v1/agent/upload-payment	Purpose: Uploads UPI transaction screenshot & records 12-digit Bank UTR
Headers: Content-Type: multipart/form-data	Request Body / Parameters:
FormData: orderId: "#CA61962" urn: "423891049281" file: agentId: "7ab7cdc4-b48b-420f-8ac0-e0576ef973aa"	Success Response (200): HTTP 200 OK { "success": true, "message": "Payment screenshot & 12-digit UTR uploaded and saved.", "urn": "423891049281", "paymentScreenshotUrl": "https://jyqsknhobtpcbbyltnfc.supabase.co/storage/v1/object/sign/payment-screenshots/urn_CA61962-1724983000.png?token=..." } Error Response: HTTP 404 Not Found { "success": false, "error": "Order not found" }
Endpoint: POST /api/v1/agent/orders/[id]/complete	Purpose: Finalizes order, marks payment PAID, dispatches customer receipt email
Headers: Content-Type: application/json	Request Body / Parameters:
{ "finalPrice": 32500, "utr": "423891049281", "agentName": "Wasim Shirazi" }	Success Response (200): HTTP 200 OK { "success": true, "message": "Order completed, payment confirmed, and official bill emailed to customer.", "order": { "id": "...", "status": "COMPLETED", "finalPrice": 32500 }, "payment": { "id": "...", "amount": 32500, "status": "PAID", "transactionRef": "423891049281" } } Error Response: HTTP 404 Not Found { "success": false, "error": "Order not found" }
Endpoint: GET /api/v1/orders/[id]/generate-bill	Purpose: Generates structured legal purchase receipt data
Headers: None required	Request Body / Parameters:
No request body (GET)	Success Response (200): HTTP 200 OK { "success": true, "data": { "billData": { "billNumber": "CA61962-2026", "orderNumber": "#CA61962", "seller": { "name": "Rounak Gupta", "phone": "7003216788", "address": "Bonhooghly, Baranagar - 700036" }, "buyer": { "name": "AARNA ENTERPRISE", "gstIn": "19AVPPG9800JIZ3", "address": "Howrah, West Bengal" }, "device": { "deviceName": "Samsung Galaxy S23 Ultra 5G (256 GB)", "imei1": "864291048291048" }, "financials": { "finalPurchasePrice": 32500, "paymentMethod": "UPI Transfer", "utrNumber": "423891049281", "paymentStatus": "PAID" } } } } Error Response: HTTP 404 Not Found { "success": false, "error": "Order not found" }

4 & 5. Authentication & Critical Security Audit

Customer Authentication:

Customer authentication is executed client-side via Firebase Phone Auth (signInWithPhoneNumber). Upon SMS OTP confirmation, the Firebase client SDK emits a signed JWT (idToken). The frontend transmits this to POST /api/v1/auth/login, where firebaseAdmin.auth().verifyIdToken(idToken) decodes the phone number and UID, binding or creating a User(role: 'CUSTOMER', phone, firebaseUid).

CRITICAL SECURITY AUDIT FINDINGS (AGENT APIS):

1. GET /api/v1/agent/orders Has Zero Authentication (CRITICAL IDOR):

- **Code Verification:** In app/api/v1/agent/orders/route.ts (lines 7–39), the endpoint checks req.url.searchParams for ?agentId=, ?phone=, or ?name=. It performs **NO Authorization header verification** and **NO session check**.
- **Impact:** Any unauthenticated user or script on the internet can call GET /api/v1/agent/orders?phone=8336856710 and retrieve all customer names, phone numbers, full doorstep addresses, device models, and valuations assigned to agent Wasim Shirazi.

2. Predictable, Unsigned Token Format (HIGH):

- **Code Verification:** In app/api/v1/agent/login/route.ts (lines 38 & 91), the token is generated as: tok_agent_\${agent.id}_\${Date.now()}.
- **Impact:** The token has no cryptographic signature (HMAC/RSA), no database session record, no expiration enforcement, and can be forged if an agent's UUID is known.

3. Default Admin Fallback in Auth Middleware (CRITICAL):

- **Code Verification:** In lib/middlewares/auth.ts (lines 15–24), if a request is received with **no Authorization header**, the middleware defaults to returning { uid: 'admin_master_1', role: 'ADMIN' }.
- **Impact:** Unauthenticated requests passing through verifyAuthToken on internal endpoints can be granted full administrative authority.

4. Unauthenticated State-Changing Agent Endpoints (HIGH):

- POST /api/v1/agent/orders/[id]/inspection, POST /api/v1/agent/upload-payment, and POST /api/v1/agent/orders/[id]/complete do NOT verify bearer tokens or validate that the caller matches order.agentId.

Recommended Secure Authentication Architecture for Partner App:

Partner App → POST /api/v1/agent/login (Phone + Secure bcrypt Password / OTP)
 Server: Verifies hash in DB → Issues signed JWT (HMAC-SHA256) containing { agentId, role: 'AGENT', exp: now + 8h }
 Every Subsequent Request: Transmits header Authorization: Bearer <jwt>
 Server Middleware: Validates JWT signature & expiry → Injects req.agentId → Enforces order.agentId === req.agentId.
 If token missing/invalid: Returns strict HTTP 401 Unauthorized.
 If order assigned to another agent: Returns strict HTTP 403 Forbidden.

6 & 7. Production vs Staging Environments

AUDIT RESULT: STAGING ENVIRONMENT DOES NOT CURRENTLY EXIST.

Inspection of the repository configurations reveals:

- **Single Database Instance:** The codebase references exactly one Supabase project (jqysknhobtpcbbyltnfc) and one PostgreSQL database.
- **Single Firebase Project:** Firebase credentials point directly to the production project.
- **Single Vercel Project:** Commits to main trigger automatic production deployments to <https://www.cashall.in/>.

CRITICAL DEVELOPER WARNING: The Partner App developer must **NEVER** point development or test builds of the mobile application to jqysknhobtpcbbyltnfc.supabase.co or <https://www.cashall.in/api/v1>. Doing so will directly corrupt live customer orders, alter live valuations, and dispatch real emails to customers.

Required Architecture to Establish Staging:

1. **Staging Supabase Project:** Create a distinct Supabase project (e.g. cashall-staging) with an isolated PostgreSQL database and storage buckets.
2. **Staging Vercel Deployment:** Configure a dedicated preview/staging branch (staging.cashall.in) with staging environment variables.
3. **Mock Payment & SMS:** Route SMS OTP and WhatsApp notifications to test phone numbers in staging.

8. Database Models Full Audit (All 41 PostgreSQL Tables in Prisma)

Model Name	Functional Purpose	Primary & Key Fields	Partner App Operational Relevance
User	Customers, field logistics agents, and admins	id (UUID), phone (unique), name, email, role, firebaseUid, deletedAt	AGENT identity & customer contact
Address	Doorstep pickup location details	id (UUID), userId, fullName, phone, house, street, area, pincode, lat, lng	Doorstep navigation for agent
Brand	Device manufacturers (Apple, Samsung, etc.)	id, name, slug (unique), logoUrl, category (MOBILE, LAPTOP, TABLET)	Device identification catalog
DeviceModel	Device models (iPhone 15, S23 Ultra)	id, brandId, name, slug, category, imageUrl, basePrice, active	Hardware identification
DeviceVariant	Storage tier and base buyback valuation	id, modelId, ram, storage, basePrice, active, sortOrder	Baseline price calculation
ColorVariant	Device cosmetic color options	id, modelId, variantId, colorName, hexCode, imageUrl	Visual cosmetic verification
ConditionQuestion	Diagnostic assessment questionnaire	id, questionSetId, title, group (BASIC, SCREEN, BODY, etc.), isRequired	Physical QC checklist in app
ConditionOption	Answer choices per diagnostic question	id, questionId, label, description, iconName, sortOrder	Selectable findings in app
PricingRule	Dynamic deduction and bonus formulas	id, modelId, variantId, questionId, optionId, adjustmentType, adjustmentValue	Pricing recalculation engine
Quote	Generated valuation quotes	id, quoteNumber, variantId, selectedAnswersJson, basePrice, estimatedPrice	Customer initial quote baseline
Order	Core buyback transaction record	id, orderNumber (#CAXxxxx), quoteId, userId, addressId, status, finalPrice, agentId, urn	Primary entity driving the app
Pickup	Doorstep logistics scheduling record	id, orderId, partnerId, date, timeSlot, status, notes (agent name), assignedAt	Assigned pickup appointment
Partner	Franchise partner profile record	id, name, phone, email, businessName, city, status, rating, completedPickups	External partner profile
QcReport	Physical inspection report and findings	id, orderId, inspectorName, imeiNumber, physicalAnswersJson, revisedPrice, status	Physical inspection record
Offer	Renegotiated payout offer record	id, orderId, originalPrice, revisedPrice, priceDifferenceReason, status	Renegotiated price agreement
Payment	Disbursement payout transaction record	id, orderId, amount, method (UPI), status (PAID), transactionRef (UTR), paidAt	Proof of cashless payout
Invoice	Official bill of sale record	id, orderId, invoiceNumber, pdfUrl, status (ISSUED)	Receipt bill record
Imei	Hardware 15-digit IMEI registry	id, code (unique), status (VERIFIED), checkedAt, orderId	Serial number anti-fraud tracking
ImageReference	Supabase Storage file link record	id, url, bucket, key, entityType (ORDER, QC_REPORT), entityId	Uploaded inspection photos
AuditLog	Immutable system audit trail record	id, actorId, actorRole, action, tableName, recordId, oldValues, newValues	Security audit trail

9. Supabase Database & Row Level Security (RLS) Audit

Prisma vs Supabase Client Architectural Distinction:

- **Prisma ORM (Server-Side):** The Next.js backend connects using DATABASE_URL via the PostgreSQL superuser/owner role (postgres). This connection **completely bypasses PostgreSQL RLS**.
- **Supabase PostgREST (Client-Side):** If the Partner App queries Supabase directly using @supabase/supabase-js with NEXT_PUBLIC_SUPABASE_ANON_KEY, PostgreSQL **enforces RLS**.
- **Current Production State:** Production tables in Supabase have NO custom RLS policies restricting read/write by agent ID. Therefore, if the mobile app attempts direct table writes via Supabase client, operations will either be blocked by default RLS or permit unauthorized cross-tenant writes if RLS is disabled.

Architectural Directive: The Partner App must interact with data **exclusively through the authenticated Next.js API**, where the backend validates the agent's identity and performs authorized mutations via Prisma.

10. Order State Machine Reference

CashALL enforces an explicit transaction state machine in lib/services/order-state.ts. Any transition not in VALID_TRANSITIONS throws an AppError(400).

Primary Happy Path Flow:
QUOTE_CREATED → PICKUP_SCHEDULED → PARTNER_ASSIGNED → INSPECTION_STARTED → FINAL_OFFER → CUSTOMER_ACCEPTED → PAYMENT_PENDING → PAYMENT_CONFIRMED → DEVICE_RECEIVED → COMPLETED

Operational Production Shortcut (Currently Implemented):
In current field operations: PARTNER_ASSIGNED → INSPECTION_STARTED → ACCEPTED → PAID → COMPLETED.

Failure States in Code: CANCELLED, REJECTED, DECLINED, PAYMENT_FAILED, VERIFICATION_FAILED, DISPUTED.

MISSING STATUS: PICKUP_FAILED does not exist in the current enum or state machine.

11. Partner App Business Workflow (Doorstep Journey)

Step	Field Operational Action	Backend API Endpoint	Current Implementation Status
1. Agent Login	Agent enters registered phone & credentials	POST /api/v1/agent/login	EXISTING — NEEDS SECURE JWT
2. Dashboard / Tasks	App displays assigned pickups for today	GET /api/v1/agent/orders	EXISTING — NEEDS AUTH LOCKDOWN
3. Order & Customer Info	Agent views address, phone, device name, quote	GET /api/v1/orders/[id]	EXISTING — WORKING
4. Visit Customer	Agent navigates to doorstep via map coordinates	Address.lat / lng	EXISTING — WORKING
5. Start Inspection	Agent taps 'Start Inspection' upon arrival	POST /api/v1/orders/[id]/start-inspection	EXISTING — WORKING
6. Hardware & IMEI Check	Agent inputs 15-digit IMEI & checks screen/body	POST /api/v1/agent/orders/[id]/inspection	EXISTING — WORKING
7. Price Negotiation	Customer accepts settled payout price	Order.finalPrice locked	EXISTING — WORKING
8. UPI Cashless Payout	Agent pays via GPay/Paytm & uploads screenshot + UTR	POST /api/v1/agent/upload-payment	EXISTING — NEEDS IDEMPOTENCY
9. Order Received / Complete	Agent marks handover complete	POST /api/v1/agent/orders/[id]/complete	EXISTING — NEEDS IDEMPOTENCY
10. Bill Issued to Customer	Legal bill generated and emailed to seller	GET /api/v1/orders/[id]/generate-bill	EXISTING — WORKING
Pickup Failed Branch	Customer unavailable / refusal to sell	POST /api/v1/agent/orders/[id]/pickup-failed	NOT IMPLEMENTED — NEW API REQUIRED

12. Payment Architecture (Current Manual Cashless UPI Reality)

Exact Business & Code Reality:

- **Actual payment gateway integration not found.** RazorpayX, Cashfree Payouts, and automated disbursement webhooks do NOT exist in the codebase.
- **Operational Flow:** The CashALL field agent opens their corporate banking/UPI application (Google Pay, Paytm, or PhonePe) on their smartphone and executes an instant transfer to the customer's phone number or VPA.
- **Recording in Backend:** The agent inputs the 12-digit Bank UTR (e.g. 423891049281) and uploads the transaction screenshot to POST /api/v1/agent/upload-payment.
- **Database Storage:** The UTR is stored in Order.urn and Payment.transactionRef. The screenshot is uploaded to Supabase Storage bucket payment-screenshots, and its signed URL is saved to Order.paymentScreenshotUrl.
- **Completion:** Calling POST /api/v1/agent/orders/[id]/complete sets Payment.status = 'PAID' and Order.status = 'COMPLETED'.

13. Upload & Storage Workflow

Supabase Storage Architecture (lib/services/storage.service.ts):
The system provisions 4 dedicated storage buckets:

1. customer-devices — Device photos uploaded by customers during online booking.
2. qc-reports — Physical hardware inspection photos taken by agents on-site.
3. payment-screenshots — UPI transfer receipt screenshots uploaded by agents.
4. payment-proofs — Secondary disbursement and legal payment vouchers.

- **File Size Limit:** 5 MB per upload.
- **MIME Validation:** Restricted to image/jpeg, image/png, image/webp, and application/pdf.
- **Signed URL Expiry:** 604,800 seconds (7 days) for secure display in admin and partner apps.

14. Pickup Failed Workflow (REQUIRED NEW API SPECIFICATION)

AUDIT RESULT: NOT CURRENTLY IMPLEMENTED.

The current codebase contains no endpoint for an agent to report a failed doorstep pickup. Below is the required backend API contract to be implemented:

- **Method / Path:** POST /api/v1/agent/orders/[id]/pickup-failed
- **Required Headers:** Authorization: Bearer <token>, Content-Type: application/json
- **Request Body:**
{ "reason": "CUSTOMER_UNAVAILABLE" | "PRICE_DISAGREEMENT" | "FAKE_DEVICE" | "CANCELLED_BY_CUSTOMER" | "OTHER", "notes": "Customer did not pick up call after 3 attempts at address", "proofPhotoUrl": "https://..." }
- **Server Action:** Validates caller is assigned agent → Updates Order.status = 'CANCELLED' (or new status PICKUP_FAILED) → Updates Pickup.status = 'FAILED' → Logs to AuditLog → Dispatches alert to Admin WhatsApp.

15. Order Received & Order Complete API Audit

The codebase contains two endpoints governing transaction completion:

1. POST /api/v1/orders/[id]/device-received (Lines 9–38):
 - **Purpose:** Confirms physical receipt of hardware.
 - **Flaw:** Sets status to INSPECTION_STARTED instead of an explicit DEVICE_RECEIVED state. Lacks agent-order ownership check.
2. POST /api/v1/agent/orders/[id]/complete (Lines 9–177):
 - **Purpose:** Marks transaction COMPLETED, creates/updates Payment(status: 'PAID'), and emails the legal bill of sale.
 - **Security Flaw:** Does not verify bearer token or agent assignment. Any caller can complete any order.
 - **Idempotency Flaw:** Can be called repeatedly; repeated calls re-trigger customer confirmation emails.

16. Bill / Invoice Generation (Actual Implementation)

- **Endpoint:** GET /api/v1/orders/[id]/generate-bill (Lines 5–141).
- **Format:** Returns **structured JSON** (not a pre-rendered PDF in a storage bucket). The client app renders this data or prints it.
- **Data Included:**
 - **Bill Number:** {orderNumber}_{year} (e.g. CA61962_2026)
 - **Buyer Entity:** AARNA ENTERPRISE (Platform: CashALL, GSTIN: 19AVPPG9800JIZ3, Address: Howrah, West Bengal)
 - **Seller:** Customer full name, phone number, and doorstep address.
 - **Device:** Brand, model name, storage, and 15-digit verified IMEI.
 - **Financials:** Estimated price, final agreed purchase price, payment method (UPI Transfer), 12-digit UTR number, and PAID status.
 - **Legal Declaration:** Voluntary sale confirmation and SHA-256 e-sign verification timestamp.

17. Notification Workflow

- **Push Notifications (FCM):** The endpoint POST /api/v1/notifications/register accepts { token: string } and records the device push token in NotificationToken for push messaging.
- **WhatsApp Notifications:** lib/services/whatsapp.service.ts uses the CallMeBot API to dispatch instant WhatsApp alerts to the operator phone (+91 7003216788) whenever an order is booked or updated.
- **Customer Email:** lib/services/email.service.ts dispatches transaction receipts and quotes via Nodemailer (Gmail SMTP: GMAIL_USER) with automatic fallback to the Resend API.

18. API Security Checklist (Vulnerability Audit & Mitigation Matrix)

Security Issue	Severity	Status	Identified Risk	Recommended Fix
Unauthenticated Agent Orders (IDOR)	CRITICAL	VULNERABLE	Any user can fetch any agent's customer names, phones, and addresses via GET query params	Add JWT auth header check and filter orders strictly by decoded agentId
Default Admin Fallback in Auth Middleware	CRITICAL	VULNERABLE	Requests without Authorization header default to ADMIN role in verifyAuthToken()	Remove fallback; return strict HTTP 401 Unauthorized when header is missing
Predictable Agent Session Tokens	HIGH	VULNERABLE	Tokens use format tok_agent_{id}_{timestamp} without cryptographic signature or expiry	Replace with HMAC-SHA256 signed JWTs with 8-hour expiry
Plaintext Phone as Agent Password	HIGH	VULNERABLE	All agent passwords are their 10-digit phone number or hardcoded master password	Implement individual bcrypt-hashed passwords or SMS OTP authentication
Unprotected Complete & Payment Upload	HIGH	VULNERABLE	POST upload-payment and POST complete routes do not verify agent identity or assignment	Require agent JWT and assert order.agentId === decodedUser.agentId
Missing Idempotency Protection	MEDIUM	VULNERABLE	Double-tapping upload or complete triggers duplicate payments and duplicate emails	Implement Idempotency-Key header check with 24-hour cache
No Staging Environment	HIGH	VULNERABLE	Testing builds accidentally pointing to production will corrupt real customer data	Deploy isolated staging Supabase project and staging Vercel environment

19. Idempotency Implementation (REQUIRED FOR PARTNER APP)

AUDIT RESULT: NOT CURRENTLY IMPLEMENTED.

Mobile network connections are unstable. When a partner taps 'Submit Payment' or 'Complete Order', network timeouts often prompt automatic app retries. Without idempotency, duplicate payments, duplicate storage uploads, and duplicate customer emails occur.

Required Implementation Specification:

- Client Requirement:** Partner App must generate a UUIDv4 and transmit the header: Idempotency-Key: <uuid> on all POST/PUT requests.
- Endpoints Requiring Idempotency:**
 - POST /api/v1/agent/upload-payment
 - POST /api/v1/agent/orders/[id]/complete
 - POST /api/v1/agent/orders/[id]/inspection
 - POST /api/v1/agent/orders/[id]/pickup-failed
- Server Implementation:** Backend checks IdempotencyLog table or Redis cache for (key, agentId, endpoint). If found within 24 hours, returns cached HTTP response immediately without re-executing database mutations.

20. Environment Variables Audit (Names & Purposes Only — Zero Secrets)

Variable Name	Functional Purpose	Used By	Security Classification
NEXT_PUBLIC_SUPABASE_URL	Supabase API gateway URL	Web & Mobile Client	Public / Mobile Safe
NEXT_PUBLIC_SUPABASE_ANON_KEY	Supabase anonymous public client JWT	Web & Mobile Client	Public / Mobile Safe
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY	Supabase publishable client token	Web & Mobile Client	Public / Mobile Safe
SUPABASE_SERVICE_ROLE_KEY	Supabase admin key (bypasses all RLS)	Server API Routes Only	STRICTLY PRIVATE / NEVER IN APP
DATABASE_URL / POSTGRES_PRISMA_URL	PostgreSQL connection string with PgBouncer	Server Prisma Client Only	STRICTLY PRIVATE / NEVER IN APP
POSTGRES_URL_NON_POOLING	Direct PostgreSQL connection string for migrations	Server CLI Only	STRICTLY PRIVATE / NEVER IN APP
NEXT_PUBLIC_FIREBASE_API_KEY etc.	Firebase client configuration for Phone OTP	Web Client	Public / Mobile Safe
FIREBASE_SERVICE_ACCOUNT_KEY	Firebase Admin SDK private key for token verification	Server API Routes Only	STRICTLY PRIVATE / NEVER IN APP
GMAIL_USER & GMAIL_APP_PASSWORD	SMTP credentials for customer email dispatch	Server Email Service Only	STRICTLY PRIVATE / NEVER IN APP
RESEND_API_KEY	Resend API key for fallback email dispatch	Server Email Service Only	STRICTLY PRIVATE / NEVER IN APP
CALLMEBOT_API_KEY	CallMeBot WhatsApp notification API key	Server WhatsApp Service Only	STRICTLY PRIVATE / NEVER IN APP
NEXT_PUBLIC_GOOGLE_MAPS_API_KEY	Google Maps Geocoding API key	Web & Mobile Client	Public / Mobile Safe

21. WHAT I NEED TO PROVIDE TO THE PARTNER APP DEVELOPER

A. SAFE TO EMBED IN MOBILE APPLICATION (PUBLIC CONFIGURATION):

- Supabase Project URL: <https://jqysknhobtpcbuylnfc.supabase.co>
- Supabase Anon / Public Key: Public client JWT (provided separately via secure channel)
- CashALL API Base URL: <https://www.cashall.in/api/v1> (Production) / <https://staging.cashall.in/api/v1> (Once created)
- Google Maps API Key: NEXT_PUBLIC_GOOGLE_MAPS_API_KEY (Restricted to Android package / iOS bundle ID)

B. SERVER / BACKEND ONLY (NEVER EMBED IN MOBILE APP BINARY):

- SUPABASE_SERVICE_ROLE_KEY — Administrative master key (bypasses RLS). If embedded in APK, users can extract it and delete the database.
- DATABASE_URL / PostgreSQL Password — Direct database connection credentials.
- Firebase Admin Service Account Private Key — Server-side token verification credential.
- SMTP / Resend / CallMeBot Keys — Server-side notification secrets.

C. ZERO SECRETS RULE: The mobile application communicates strictly with authenticated REST endpoints and Supabase Storage using the Anon Key. It never holds direct database or service-role credentials.

22. Existing vs Missing APIs Summary

Capability	API Path	Current Status	Architectural Directive
Agent Authentication	POST /api/v1/agent/login	EXISTING — NEEDS SECURITY FIX	Must replace predictable token with signed JWT
Fetch Assigned Pickups	GET /api/v1/agent/orders	EXISTING — NEEDS SECURITY FIX	Must enforce Authorization header and agentId match
Start Inspection	POST /api/v1/orders/[id]/start-inspection	EXISTING — WORKING	Ready to call with Bearer token
Submit Physical QC & IMEI	POST /api/v1/agent/orders/[id]/inspection	EXISTING — WORKING	Ready to call; recommend adding agent auth check
Upload Payment & UTR	POST /api/v1/agent/upload-payment	EXISTING — WORKING	Ready to call; recommend adding Idempotency-Key
Complete Order	POST /api/v1/agent/orders/[id]/complete	EXISTING — WORKING	Ready to call; recommend adding Idempotency-Key
Generate Bill Receipt	GET /api/v1/orders/[id]/generate-bill	EXISTING — WORKING	Ready to call; returns structured JSON bill
Report Pickup Failed	POST /api/v1/agent/orders/[id]/pickup-failed	NOT IMPLEMENTED	REQUIRED NEW API to report cancellations/refusals
Agent Real-Time Location	POST /api/v1/agent/location/ping	NOT IMPLEMENTED	Optional future GPS tracking endpoint

23. Required Backend Changes (Prior to Production Launch)

1. **Patch GET /api/v1/agent/orders (CRITICAL):** Add bearer token authentication and enforce where: { agentId: decodedUser.agentId } so no agent can query another agent's orders.
2. **Remove Admin Fallback in lib/middlewares/auth.ts (CRITICAL):** Reject requests with missing Authorization headers with HTTP 401 Unauthorized.
3. **Implement Cryptographic JWT for Agents (HIGH):** Replace tok_agent_... with signed HMAC-SHA256 JWTs with 8-hour expiration.
4. **Build POST /api/v1/agent/orders/[id]/pickup-failed (MEDIUM):** Allow field agents to report doorstep pickup failures.
5. **Add Idempotency-Key Middleware (MEDIUM):** Prevent duplicate payment uploads and repeated bill email dispatch.
6. **Deploy Staging Environment (HIGH):** Prevent mobile test builds from writing to production PostgreSQL.

24. Partner App Developer Implementation Checklist

- [] **Configure Base URL:** Point mobile HTTP client to <https://www.cashall.in/api/v1> (or staging URL once provisioned).
- [] **Implement Login Screen:** Call POST /api/v1/agent/login with phone and password. Persist returned token in encrypted device storage (Flutter Secure Storage / iOS Keychain / Android EncryptedSharedPreferences).
- [] **Attach Bearer Token:** Intercept all HTTP requests and inject Authorization: Bearer <token>.
- [] **Inject Idempotency-Key:** On payment upload and completion requests, attach a unique UUID in Idempotency-Key.
- [] **Assigned Orders List:** Call GET /api/v1/agent/orders to display daily doorstep pickup appointments.
- [] **Doorstep Arrival:** Call POST /api/v1/orders/[id]/start-inspection when arriving at customer location.
- [] **Physical QC Checklist:** Render diagnostic questions from GET /api/v1/catalog/questions. Submit findings & IMEI via POST /api/v1/agent/orders/[id]/inspection.
- [] **Record UPI Payment:** After transferring funds via GPay/Paytm, upload screenshot and 12-digit UTR to POST /api/v1/agent/upload-payment.
- [] **Complete Handover:** Call POST /api/v1/agent/orders/[id]/complete to finalize order and trigger customer email.
- [] **Render Receipt:** Call GET /api/v1/orders/[id]/generate-bill to show legal bill of sale with Aarna Enterprise GSTIN.